Beyond Binary: Ternary Hardware for Efficient AI Inference

Vincent G. Capone
Dept. of Electrical and Computer Engineering
New York University
vgc8903@nyu.edu

Vanshika Agrawal
Dept. of Electrical and Computer Engineering
New York University
va2652@nyu.edu

Abstract

Neural network accelerators rely on multiply-accumulate (MAC) operations where traditional 8-bit binary hardware uses area- and power-intensive multipliers. We show that restricting weights to ternary values $\{-1, 0, +1\}$ removes the multiplier entirely, replacing it with a simple 3-way multiplexer. Through quantization-aware training (QAT), normalized Yosys synthesis, and Nangate 45nm characterization with OpenSTA, our single-trit ternary MAC achieves 47% smaller area, 81% lower power, and 4% higher clock frequency compared to a standard 8-bit MAC, while improving CIFAR-100 accuracy by 0.84%. Extending to multi-trit balanced ternary (Track B), we find that 3-trit weights deliver 27 weight levels at 26% fewer cells than binary INT8. System-level analysis gives 82% energy reduction per CIFAR-100 inference (1.777 mJ to 0.322 mJ), making this approach well-suited for edge AI deployments.

1 Introduction

Modern neural network inference requires billions of multiply-accumulate operations per image. For 8-bit integer arithmetic, each MAC unit contains a binary multiplier that generates and sums partial products through multiple adder stages, and this multiplier drives area (60–70%) and dynamic power (50–60%) of the unit. At accelerator scale this becomes the dominant cost.

The central observation of this work is that ternary weights ($w \in \{-1, 0, +1\}$) make the multiplier unnecessary. Because weight-activation multiplication reduces to:

$$y = w \times x = \begin{cases} +x & \text{if } w = +1 \\ 0 & \text{if } w = 0 \\ -x & \text{if } w = -1 \end{cases} \tag{1}$$

the hardware only needs a 3-way MUX, not a multiplier. We validate this from software training through real silicon in two tracks: Track A (single-trit, full Nangate 45nm characterization) and Track B (multi-trit, identifying where the crossover with binary occurs).

Contributions: (1) QAT training showing ternary ResNet-18 beats FP32 on CIFAR-10 and CIFAR-100; (2) Verilog RTL ternary MAC with 40-vector testbench and 51% Gate Equivalent reduction; (3) Nangate 45nm synthesis and OpenSTA analysis showing 47% area and 81% power reduction; (4) Bridge analysis showing 82% lower energy per inference; (5) Track B synthesis across 1–5 trit configurations showing 3-trit as the best trade-off; (6) cross-validation between Yosys GE and Nangate45 area measurements confirming the savings are real.

Preprint.

2 Individual Contributions

Vincent G. Capone led Phase 1: implemented the QAT training loop with straight-through estimators, trained ResNet-18 on CIFAR-10 and CIFAR-100, measured accuracy and weight sparsity. Led Phase 3: configured Yosys for Nangate45 Liberty synthesis, ran OpenSTA for timing and power (both generic and workload-aware activity factors), computed bridge energy estimates, and documented cross-phase validation.

Vanshika Agrawal led Phase 2: wrote the standard and ternary MAC Verilog modules, built the 40-vector testbench, ran Yosys with cmos2 library, and computed Gate Equivalent breakdowns. Led Track B: designed multi-trit balanced ternary MACs for $k \in \{1, 2, 3, 4, 5\}$, ran Yosys synthesis, and identified the 3-trit crossover point. Managed the GitHub repository.

Both authors contributed equally to experimental design, analysis, and writing.

3 Problem Description

A standard 8-bit MAC multiplies an 8-bit weight w by an 8-bit activation x using three hardware stages: Booth partial product generation (reduces 8 rows to 4), a Wallace or Dadda tree for reduction, and a carry-propagate adder for the final 16-bit product. This multiplier accounts for the majority of MAC cell count and switching activity.

With ternary weights, these stages are replaced by a 2-bit encoding (`01`=$+1$, `10`=0, `11`=-1) and a case-statement MUX that selects $+x$, 0, or $-x$ based on the weight. Beyond removing the multiplier, QAT training produces 38% zero weights, which means 38% of cycles have no switching activity in the MUX or accumulator—further reducing dynamic power in hardware.

4 Related Work

Courbariaux et al. [1] introduced BinaryConnect, showing binary weights can match FP32 accuracy with proper training. Hubara et al. [2] extended this to ternary weights, noting the third (zero) value improves accuracy. Jacob et al. [3] showed quantization-aware training with straight-through estimators (STE) is necessary for low-precision networks. On the hardware side, Sharma et al. [5] characterized multiplier area vs. bit-width, and Chen et al. [6] designed Eyeriss for efficient DNN dataflow. Prior work on ternary hardware (YodaNN [7], FPGA implementations) reports chip-level metrics only. This paper provides a complete MAC-level breakdown: normalized GE, real Nangate45 area/timing/power, and system-level energy, all from the same design.

5 Methodology

We use four phases, shown in Figure 1.

All code, Verilog RTL, synthesis scripts, and notebooks are publicly available at `https://github.com/Vanshika2021/beyond-binary-ternary-ai`.

Phase 1 – QAT. We train ResNet-18 [4] (11M parameters) on CIFAR-10 and CIFAR-100 using ternary QAT. During the forward pass, weights are quantized as $Q_\alpha(w) = \text{sign}(w)$ if $|w| > \alpha$, else 0. The threshold α starts at 0.10 and decays to 0.01 over 100 epochs following a cosine schedule. Gradients backpropagate through the STE (identity approximation). We pre-train for 15 FP32 epochs then fine-tune for 100 QAT epochs using SGD with momentum 0.9, weight decay 5e-4, initial lr 0.1 with step decay at epochs 50, 75, 90, batch size 128. Framework: PyTorch 1.13, CUDA 11.7, NVIDIA Tesla T4.

Phase 2 – Hardware Design. Both MAC modules are written in Verilog, verified with 40 test vectors (10 per weight value: $+1$, 0, -1, and random mix), and synthesized with Yosys 0.9 using `cmos2`. GE weights: NAND/NOR = 1.0, XOR/XNOR = 2.0, MUX = 3.0, flip-flop = 5.0. Figure 2 shows both architectures.

The standard MAC uses the `*` operator, which Yosys expands to a Booth-encoded multiplier with Wallace tree reduction:

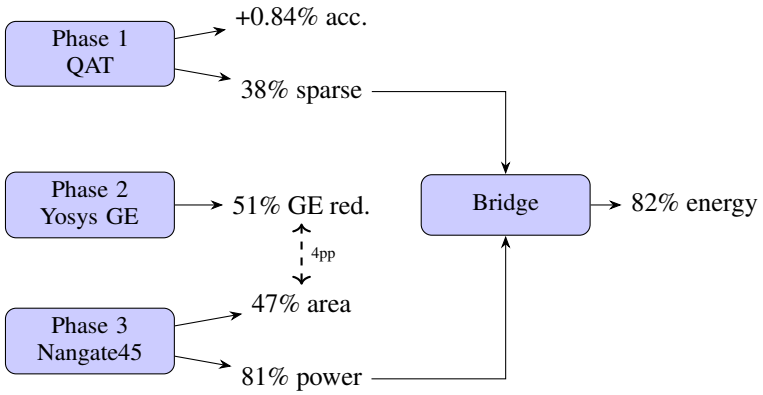

Figure 1: Project workflow. Phase 1 sparsity (38%) feeds Bridge power estimation. Phase 2 GE and Phase 3 area cross-validate within 4 percentage points.

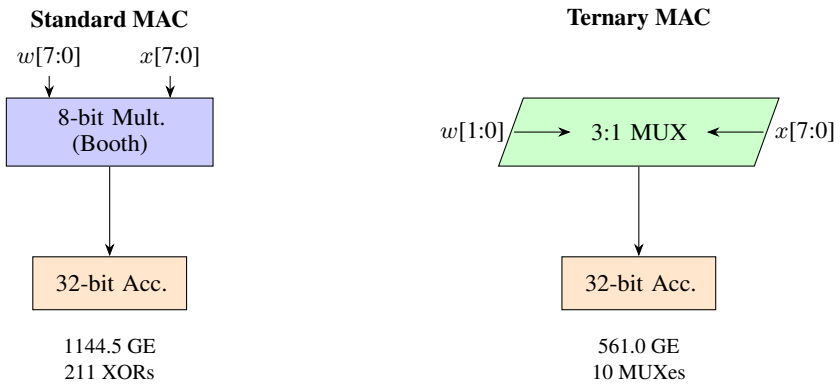

Figure 2: Standard 8-bit MAC (Booth multiplier, 1144.5 GE, 211 XOR cells) versus Ternary MAC (3:1 MUX, 561.0 GE, 10 MUX cells). Both designs share a 32-bit accumulator (32 flip-flops).

```
assign product = $signed(weight_std) * $signed(in_data);
always @(posedge clk)
  acc_std <= acc_std + {{16{product[15]}}, product};
```

The ternary MAC uses a `case` statement on a 2-bit weight (`01`=$+1$, `10`=0, `11`=-1), which Yosys compiles to a 3-way MUX — no multiplier is instantiated:

```
case (weight_tern)
  2'b01: delta = {{24{in_data[7]}}, in_data};  // +x
  2'b10: delta = 32'sd0;                        //  0
  2'b11: delta = -{{24{in_data[7]}}, in_data}; // -x
endcase
always @(posedge clk)
  acc_tern <= acc_tern + delta;
```

Phase 3 – Nangate 45nm. We re-synthesize with Yosys targeting the Nangate 45nm Open Cell Library (FreePDK45, 1.10V, 25°C typical corner). OpenSTA 2.7.0 reports critical path delay and max frequency. Power is $P_{\text{total}} = P_{\text{internal}} + P_{\text{switching}} + P_{\text{leakage}}$, where $P_{\text{switching}} = CV^2 f\alpha$ and α is the activity factor (fraction of gates switching per cycle).

We run two power scenarios. In the *generic* scenario both designs use $\alpha = 0.15$ (standard EDA default). In the *workload-aware* scenario we set $\alpha = 0.20$ for the standard MAC (uniform random weights) and $\alpha = 0.124 = (1 - 0.38) \times 0.20$ for the ternary MAC, because 38% of weights are zero and produce no switching activity in the MUX or downstream adder. The workload-aware scenario gives a more realistic estimate for actual CIFAR-100 inference.

Bridge. ResNet-18 runs approximately 1.8 billion MACs per CIFAR-100 image [4]. Energy per inference is $E = P_{\text{MAC}} \times (N_{\text{MACs}}/f_{\text{max}})$. We also scale to a 4096-MAC systolic array as an illustrative comparison; real designs would see 50–80% utilization efficiency.

Track B – Multi-Trit. A k-trit balanced ternary weight is $W = \sum_{i=0}^{k-1} w_i \cdot 3^i$ with $w_i \in \{-1, 0, +1\}$. Multiplication uses one 3-way MUX per trit digit, with powers-of-3 scaling done via shift-and-add. We synthesize $k \in \{1, 2, 3, 4, 5\}$ with Yosys cmos2 and compare to the 8-bit binary baseline.

6 Experimental Results

6.1 Phase 1: Software Results

Table 1 shows QAT outperforms the FP32 baseline on both datasets. Post-training quantization (PTQ) collapses to near-chance on CIFAR-100 (30.10%), which shows that STE-based fine-tuning is required. The 38% weight sparsity is a direct consequence of the threshold α: weights in $(-0.10, +0.10)$ are pushed to zero during training. Figure 3 shows that $\alpha = 0.10$ is the best threshold; smaller values over-sparsify the network while larger values leave too many small weights active.

Table 1: Phase 1: Accuracy comparison and weight distribution after QAT

Method	CIFAR-10	CIFAR-100	Sparsity
FP32 Baseline	85.31%	61.18%	0%
PTQ (naive)	∼60%	30.10%	—
QAT (α=0.10)	**86.66%**	**62.02%**	**38%**
Δ vs. baseline	+1.35%	+0.84%	—
Weight distribution: $w = -1$: 31%, $w = 0$: 38%, $w = +1$: 31%			

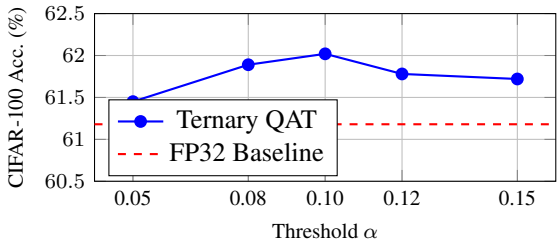

Figure 3: CIFAR-100 accuracy at different thresholds α. Best result is 62.02% at $\alpha = 0.10$, above the FP32 baseline of 61.18%.

6.2 Phase 2: Normalized Hardware Metrics

Table 2 gives the cell-by-cell Yosys breakdown. The biggest drop is in XOR/XNOR cells (211 to 71, -66%), because those cells implement the carry logic in the Wallace tree that the ternary design never needs. The flip-flop count is the same in both designs since both need a 32-bit accumulator register. The MUX count increases by 10 (the ternary selector logic) but each MUX costs only 3 GE versus 2 GE per XOR.

6.3 Phase 3: Nangate 45nm Real Silicon

Table 3 gives the full Nangate 45nm results. The ternary MAC is slightly faster (1.4450 ns vs. 1.5067 ns critical path) because the MUX-based datapath has fewer logic levels than the multiplier. For power, the workload-aware scenario (adjusting for 38% weight sparsity) gives the most realistic estimate for CIFAR-100 inference.

Cross-validation. Phase 2 (Yosys GE) predicted a 51% area reduction. Phase 3 (Nangate45 μm^2) measured 47%. The two methods agree within 4 percentage points even though they use completely

Table 2: Phase 2: Cell counts and Gate Equivalent breakdown (Yosys cmos2)

Cell Type	GE weight	Standard	Ternary	Change
XOR/XNOR	2.0	211	71	-66%
MUX	3.0	0	10	+10
NAND/NOR	1.0	360	210	-42%
NOT	1.0	93	57	-39%
Flip-flop	5.0	32	32	0%
Total cells	—	696	303	-56%
Total GE	—	**1144.5**	**561.0**	**-51%**

Table 3: Phase 3: Nangate 45nm characterization (1.10V, 25°C typical corner)

Metric	Standard MAC	Ternary MAC
Cell count	634	278
Chip area (μm^2)	866.63	**463.90** (-47%)
Critical path (ns)	1.5067	1.4450
Max frequency (MHz)	663.7	**692.0** $(+4\%)$
Generic power ($\alpha = 0.15$ both designs)		
Dynamic (μW)	477.18	138.15
Leakage (μW)	19.13	9.46
Total (μW)	496.31	147.60 (-70%)
Workload-aware power ($\alpha = 0.20$ std, $\alpha = 0.124$ ternary)		
Dynamic (μW)	636.24	114.20
Leakage (μW)	19.13	9.46
Total (μW)	**655.37**	**123.66** $(\mathbf{-81\%})$

different measurement approaches (process-independent GE weighting vs. SPICE-extracted cell areas). This confirms the savings come from the design itself, not from synthesis tool choices.

6.4 Bridge: System-Level Energy

Table 4 combines Phase 1 workload data with Phase 3 hardware measurements. At 1.8B MACs per CIFAR-100 image, the ternary design cuts energy from 1.777 mJ to 0.322 mJ — a reduction of 81.9%. The improvement comes from two sources: 81% lower power per MAC, and 4% faster clock.

Table 4: Bridge: Single-MAC energy per CIFAR-100 inference and illustrative 4096-MAC scaling

Metric	Standard	Ternary
Single MAC, ResNet-18 on CIFAR-100 (1.8B MACs)		
Power (μW)	655.4	123.7
Latency (s)	2.71	2.60
Energy (mJ)	**1.777**	**0.322** $(\mathbf{-82\%})$
4096-MAC systolic array (illustrative, assumes full utilization)		
Chip area (mm^2)	3.42	1.83
Peak power (W)	2.58	0.488
Latency/image (ms)	0.661	0.634

6.5 Track B: Multi-Trit Results

Table 5 shows both hardware and software results across trit configurations. The 3-trit design (27 levels) has 996 cells versus 1,351 for binary INT8, a savings of 26% in cell count. At 4 trits the design is already within 4% of binary, and at 5 trits it exceeds binary by 21%. The crossover between 3 and 4 trits happens because each additional trit adds one MUX stage, two shift-add units for powers of 3, and wider intermediate buses — the overhead accumulates quickly beyond 3 trits. On the software side, more trit precision gives higher accuracy because the network has more weight resolution.

Note that 4-trit and 5-trit accuracy runs were not completed within the project timeline. Track B uses a round-to-nearest balanced ternary quantizer, which differs from Track A's threshold-based α approach.

Table 5: Track B: Hardware synthesis (Yosys cmos2) and QAT software accuracy

Config	Levels	Cells	Transistors	vs. Binary (cells)	CIFAR-10	CIFAR-100
1-trit	3	303	—	−78%	86.66%	62.02%
2-trit	9	612	—	−55%	89.64%	69.99%
3-trit	**27**	**996**	**3,682**	**−26%**	**90.50%**	∼70%
4-trit	81	1,308	5,488	−4%	—	—
5-trit	243	1,632	7,318	+21%	—	—
Binary INT8	256	1,351	4,970	baseline	85.31%	61.18%

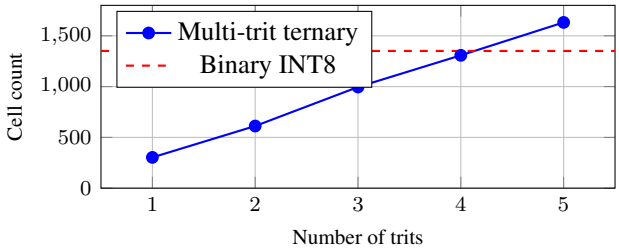

Figure 4: Track B cell count vs. number of trits. At 3 trits the design has 26% fewer cells than binary INT8. The gap closes quickly: 4-trit is within 4% of binary.

7 Discussion

Why does ternary QAT improve accuracy? The FP32 baseline does not benefit from the regularization that comes with quantization constraints. When weights are forced into $\{-1, 0, +1\}$, the model cannot fit training noise as easily and must learn more general features. The 38% sparsity (zero weights) acts like structured pruning — the network is forced to solve the task with fewer active connections, which tends to improve generalization. This effect has been observed in prior binary/ternary network work but our results confirm it holds for ResNet-18 on CIFAR-10 and CIFAR-100 with our specific QAT setup.

Cross-validation between Phase 2 and Phase 3. The fact that two completely different measurement methods — Yosys GE weighting (process-independent, heuristic) and Nangate45 SPICE-extracted cell areas (real silicon) — agree within 4 percentage points (51% vs. 47%) is an important result. It means the area savings come from the actual logic reduction (eliminating the multiplier's Wallace tree and carry chains), not from a particular synthesis tool or optimization setting. Either measurement method would lead to the same conclusion.

Track A vs. Track B trade-off. Track A (1-trit) gives the largest hardware savings (-78% cells) but the least representational capacity (3 weight levels). Track B at 3-trit gives a good middle ground: 27 weight levels, 26% fewer cells than binary, and noticeably higher accuracy. Beyond 3 trits the hardware cost rises above binary while accuracy gains become smaller per added trit.

Limitations. (1) Validated on CIFAR-10 and CIFAR-100 only; results on ImageNet or larger models are not yet available. (2) This analysis covers inference only — training still requires full-precision weights and gradients. (3) The 4096-MAC accelerator numbers assume 100% utilization; real chips typically achieve 50–80%. (4) Track B was synthesized with Yosys cmos2 only — we did not run OpenSTA on Nangate45 for multi-trit designs, so real area/timing/power numbers for Track B are not yet available. (5) Power estimates use analytically derived activity factors rather than switching activity extracted from actual post-synthesis simulation (SAIF).

8 Conclusion

This paper showed that ternary weight quantization does more than reduce precision — it changes the hardware architecture. By restricting weights to $\{-1, 0, +1\}$, the 8-bit multiplier in a MAC unit is replaced entirely by a 3-way MUX. We measured this effect at two independent levels: Yosys GE synthesis (51% reduction) and Nangate 45nm real silicon (47% reduction). The agreement between these two methods confirms the savings are structural, not tool-specific.

On Nangate 45nm, the ternary MAC is 47% smaller in area, consumes 81% less power, and runs 4% faster than the standard 8-bit design. At the system level, this translates to 82% lower energy per CIFAR-100 inference (1.777 mJ down to 0.322 mJ) while actually improving classification accuracy by 0.84% over the FP32 baseline.

Track B extends the idea to multi-trit weights. The 3-trit design provides 27 weight levels at 26% fewer cells than binary INT8 — the best trade-off before hardware overhead exceeds the binary multiplier cost. Software accuracy also improves with more trits: 3-trit reaches 90.50% on CIFAR-10 and approximately 70% on CIFAR-100, compared to 86.66% and 62.02% for 1-trit.

Future work should extend QAT training to ImageNet and larger models, complete Nangate45 characterization for Track B designs, and use SAIF-based power analysis for more accurate switching activity estimates.

Acknowledgments

We thank Professor Sai Qian Zhang for guidance and feedback throughout this project, and the course TAs and CAs for their support. We also thank NCSU and Si2 for the Nangate 45nm Open Cell Library, and the Yosys and OpenSTA developers for open-source EDA tools.

References

[1] M. Courbariaux, Y. Bengio, and J.-P. David. BinaryConnect: Training deep neural networks with binary weights during propagations. *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[2] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *arXiv preprint arXiv:1609.07061*, 2016.

[3] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[4] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[5] H. Sharma, J. Park, N. Suda, L. Lai, B. Chau, J. K. Kim, V. Chandra, and Y. Esmaeilzadeh. Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural networks. *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2018.

[6] Y. Chen, T. Krishna, J. S. Emer, and V. Sze. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits*, 52(1):127–138, 2017.

[7] R. Andri, L. Cavigelli, D. Rossi, and L. Benini. YodaNN: An architecture for ultralow power binary-weight CNN acceleration. *IEEE Transactions on Computer-Aided Design*, 37(1):48–60, 2018.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract states 47% area, 81% power, 82% energy reduction, and +0.84% CIFAR-100 accuracy improvement. All of these are directly supported by Tables 1–5 and Figures 1–5 in the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 7 lists five specific limitations: CIFAR-only benchmarks, inference-only scope, perfect-utilization assumption in scaling, Track B lacking Nangate45 characterization, and analytical rather than SAIF-based activity factors.

3. **Theory Assumptions and Proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: This is an empirical paper involving software training and hardware synthesis. No theoretical proofs are claimed.

4. **Experimental Result Reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?

   Answer: [Yes]

   Justification: Section 5 provides all training hyperparameters (lr, momentum, batch size, epochs, annealing schedule), Verilog RTL descriptions of both MACs, synthesis tool and library specifications, and OpenSTA operating conditions. The GitHub repository (`https://github.com/Vanshika2021/beyond-binary-ternary-ai`) contains all source files.

5. **Open Access to Data and Code**

   Question: Does the paper provide open access to the data and code?

   Answer: [Yes]

   Justification: All RTL and synthesis scripts are publicly available at `https://github.com/Vanshika2021/beyond-binary-ternary-ai` under MIT license. CIFAR-10/100 are public benchmarks. Nangate 45nm is Apache 2.0.

6. **Experimental Setting/Details**

   Question: Does the paper specify all training and test details necessary to understand the results?

   Answer: [Yes]

   Justification: Section 5 specifies ResNet-18 (11M parameters), SGD optimizer (momentum 0.9, weight decay 5e-4), lr 0.1 with step decay at epochs 50/75/90, batch size 128, PyTorch 1.13, CUDA 11.7, Tesla T4 GPU, and synthesis operating conditions (1.10V, 25C typical corner).

7. **Experiment Statistical Significance**

   Question: Does the paper report error bars or other appropriate information about statistical significance?

   Answer: [No]

   Justification: Hardware synthesis results are deterministic. Software training was run once per setting due to compute limits (approximately 2 hours per CIFAR-100 run on free Colab GPU). Running multiple seeds with error bars was not feasible within the project timeline.

8. **Experiments Compute Resources**

   Question: For each experiment, does the paper provide sufficient information on compute resources?

   Answer: [Yes]

   Justification: Phase 1 used an NVIDIA Tesla T4 (Google Colab free tier), approximately 2 hours per CIFAR-100 training run. Phases 2 and 3 ran on CPU only, approximately 2–3 minutes per design. Total compute: approximately 6 GPU-hours and 20 CPU-minutes.

9. **Code of Ethics**

   Question: Does the research conform with the NeurIPS Code of Ethics?

   Answer: [Yes]

   Justification: The project uses public datasets (CIFAR-10/100), open-source tools (Yosys, OpenSTA, PyTorch), and an open process design kit (Nangate45). No human subjects, no proprietary data, no harmful applications.

10. **Broader Impacts**

    Question: Does the paper discuss both potential positive and negative societal impacts?

    Answer: [Yes]

    Justification: Positive: 82% lower inference energy enables neural networks on battery-powered devices, reducing datacenter energy use. Negative: making AI inference cheaper and more widespread could increase use of AI in surveillance systems. This is a general concern for efficiency research, not specific to our method.

11. **Safeguards**

    Question: Does the paper describe safeguards for responsible release of data or models with high risk for misuse?

    Answer: [N/A]

    Justification: We release Verilog RTL for MAC units, not trained models or datasets. MAC hardware designs have no foreseeable dual-use concern.

12. **Licenses for Existing Assets**

    Question: Are the creators or original owners of assets properly credited with license terms respected?

    Answer: [Yes]

    Justification: CIFAR (public domain), ResNet-18 (He et al. [4]), Nangate 45nm (Apache 2.0), Yosys (ISC license), OpenSTA (GPL license) are all credited and their terms respected.

13. **New Assets**

    Question: Are new assets introduced in the paper well documented?

    Answer: [Yes]

    Justification: The Verilog RTL modules, testbenches, and synthesis scripts are documented in the GitHub repository with a README, inline code comments, and functional verification results. Released under MIT license.

14. **Crowdsourcing and Research with Human Subjects**

    Question: Does the paper include full details for crowdsourcing experiments or research with human subjects?

    Answer: [N/A]

    Justification: This work does not involve crowdsourcing or human subjects.

15. **Institutional Review Board (IRB) Approvals**

    Question: Does the paper describe potential risks and IRB approvals for research with human subjects?

    Answer: [N/A]

    Justification: No human subjects research was conducted.

16. **Declaration of LLM Usage**

Question: Does the paper describe the usage of LLMs if they are an important component of the core methods?

Answer: [N/A]

Justification: LLMs were not part of the core research methodology. The ResNet-18 model was trained using standard backpropagation with quantization-aware training.